Autonomous Job Application Agent

1. Description

The Autonomous Job Application Agent is a multi-agent AI pipeline that automates the full job application workflow. Given a job description and a candidate's resume, the system first validates and sanitises the resume through a **PII guardrail**, then researches the target company, extracts structured role requirements, and generates three personalised outputs in parallel: a tailored resume, a cover letter, and an interview preparation guide.

A dedicated quality-control agent scores all outputs and either auto-retries generation (up to 3 times) or routes to a **human-in-the-loop** review gate, where the candidate can approve or provide targeted per-document feedback. Approved documents are exported as markdown or PDF.

Framework	LLM	Pattern
LangGraph 0.2+ / LangChain	Configurable chat LLM	Multi-agent DAG + HITL + PII Guard

2. Architecture Overview

The system is a LangGraph **StateGraph** — a directed graph where each node is a specialised agent. All agents share a single typed state object (**AgentState**) and communicate exclusively by reading from and writing partial updates to it. The graph has five execution tiers:

Tier	Nodes	Pattern
PII Guard	pii_guardrail	First gate — blocks SSN/CC/Passport; redacts email/phone/URLs to reversible tokens before any LLM call
Entry Pipeline	parse_input → research_company → analyze_jd	Sequential — builds context chain; web_search tool used for company intel
Generation	tailor_resume, write_cover_letter, prepare_interview	Parallel fan-out — all three run concurrently; PII tokens restored in final output
Quality Gate	aggregate_outputs	Fan-in — LLM critic scores docs 0–1; auto-retries if score < 0.7 (max 3×)
HITL Review	human_review → classify_feedback	Interrupt — human approves or gives per-document feedback; classify_feedback routes to approve / regenerate / re-interrupt

3. Multi-Agent DAG — Execution Flow

The diagram below shows the full graph topology. The PII guardrail intercepts the resume before any LLM call; the orchestrator then delegates to specialised sub-agents — each bound to its own tools — and results fan in to a shared output / human-review node. Dashed arrows represent the conditional retry and feedback re-route loops.

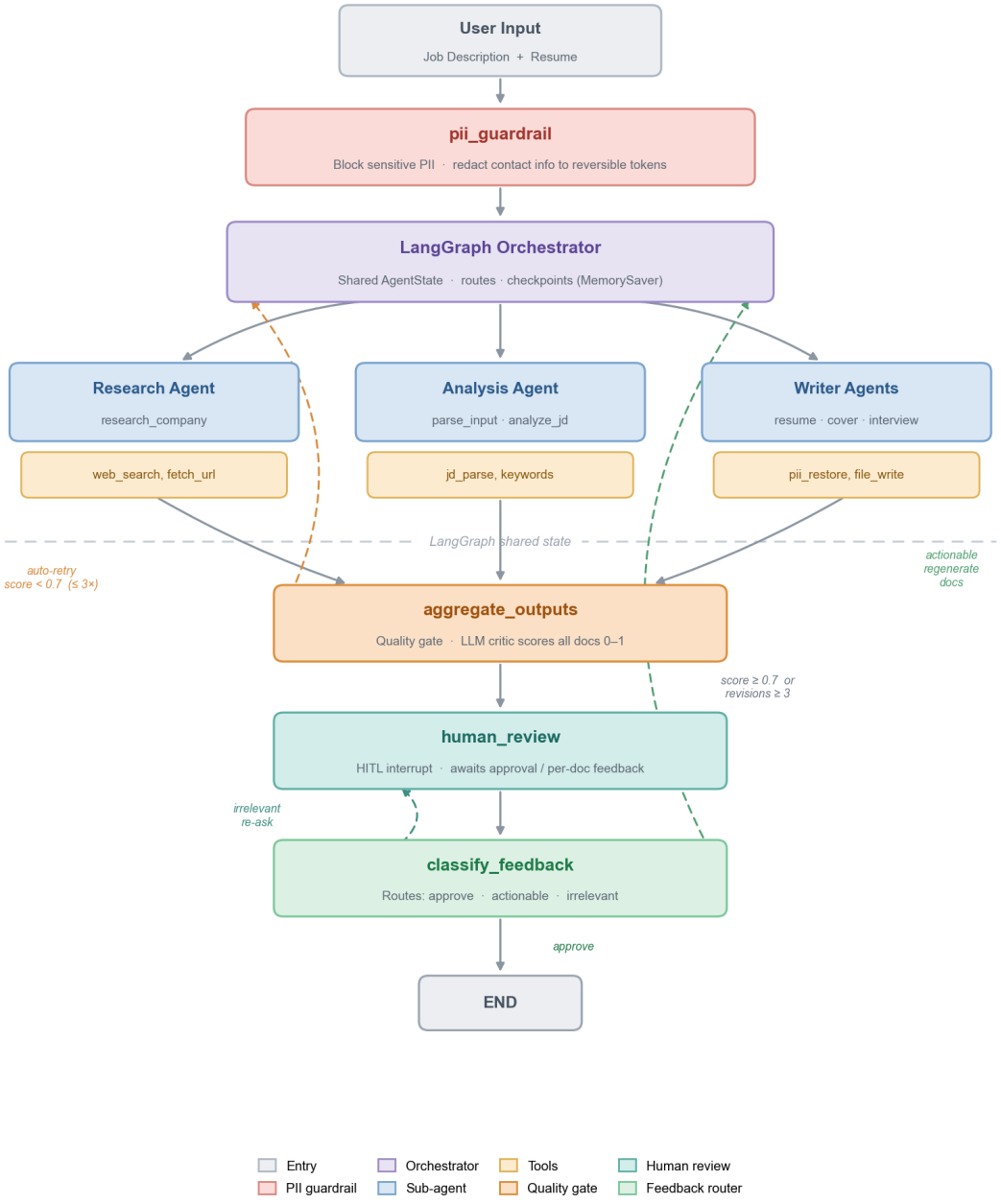

Figure 1 — LangGraph multi-agent DAG: PII guard, research / analysis / writer sub-agents, and the human-review feedback loops.

4. Data Flow

4.1 PII Guard Stage

Before any LLM call, the resume passes through **guardrails/pii_guard.py**. Sensitive PII raises a **PIIBlockedError** and halts the pipeline. Standard PII is redacted to opaque tokens; the mapping (*token → original value*) is stored in state but never logged.

PII Type	Tier	Action	Token Example
SSN, Credit Card, Passport	Sensitive	BLOCK — pipeline halts	—
Email	Standard	Redact	[PII:EMAIL:a1b2c3]
Phone	Standard	Redact	[PII:PHONE:d4e5f6]
LinkedIn / GitHub URL	Standard	Redact	[PII:LINKEDIN:g7h8i9]
Home Address	Standard	Redact	[PII:ADDRESS:j0k1l2]

4.2 Processing Stage

Node	Temp	Key Input	Key Output
parse_input	0.0	JD + resume	company_name
research_company	0.2	company_name	company_research
analyze_jd	0.0	JD	jd_analysis (skills, ATS keywords, tone)
tailor_resume	0.4	sanitized_resume + jd_analysis	tailored_resume (PII restored)
write_cover_letter	0.6	sanitized_resume + company_research	cover_letter (PII restored)
prepare_interview	0.4	sanitized_resume + jd_analysis	interview_questions
aggregate_outputs	0.0	all 3 docs + JD	quality_score, quality_feedback

4.3 PII Restoration

After each generation node, **restore_pii(output, pii_mapping)** swaps tokens back to real values. **strip_stray_pii_tokens(output)** then removes any **[PII:*]** tokens the LLM hallucinated. The LLM never sees real contact details; final documents have them fully restored.

4.4 HITL & Feedback Routing

At the **human_review** interrupt the web UI surfaces three document cards with quality scores. The user can approve globally or submit per-document feedback. **classify_feedback** then routes:

Signal	Detection	Route
Explicit approval flag	approved=True from UI	→ END
Per-document feedback fields populated	UI submit with text	→ regenerate only changed docs
Free-text < 8 chars or vague	Keyword + LLM	→ re-interrupt (ask again)

5. Quality Assurance

5.1 In-Agent Quality Loop

The **aggregate_outputs** node runs an LLM critic (temp = 0.0). Scores below 0.7 trigger an automatic retry (max 3×); after that the graph always proceeds to human review regardless of score.

Condition	Route
quality_score ≥ 0.7	→ human_review
quality_score < 0.7 AND revisions < 3	→ auto-retry (back to generation tier)
revisions = 3 (regardless of score)	→ human_review

5.2 External Evaluation Suite (evals/)

Metric	Threshold	What it checks
No Fabrication (GEval)	0.8	Every claim in tailored resume traces to original resume
JD Alignment (GEval)	0.8	Resume matches JD keywords and requirements
Cover Letter Quality (GEval)	0.8	Specificity, tone match, no clichés
Faithfulness (GEval)	0.8	Cover letter grounded only in resume facts
Fabrication Regression	Must pass	Invented employer must score LOW; honest resume must PASS

6. State Schema

All pipeline data flows through a single **AgentState TypedDict**. The **pii_mapping** field is held in memory only and is never written to logs.

Field	Type	Purpose
job_description / resume_text	str	Raw user inputs
pii_report	dict	{type: count} — safe to log
pii_mapping	dict	{token: original} — never logged
sanitized_resume	str	Redacted resume for all LLM calls
company_name / company_research	str	From parse_input / research_company
jd_analysis	dict	Skills, ATS keywords, tone, seniority
tailored_resume / tailored_resume_sanitized	str	Final (PII restored) / LLM draft for revision context
cover_letter / interview_questions	str / list	Final outputs (PII restored where applicable)
quality_score / quality_feedback	float / str	LLM critic score + narrative feedback
revision_count	int	Auto-retry counter (max 3)
human_feedback / feedback_type / approved	str / str / bool	HITL control fields
resume_feedback / cover_letter_feedback / interview_feedback	str	Per-document feedback from UI

7. Technology Stack

Layer	Technology	Role
LLM	Configurable chat LLM (via LangChain)	Inference for all 8 nodes (temp 0.0–0.6 per node)
Orchestration	LangGraph (StateGraph + MemorySaver)	Agent DAG + HITL checkpoint / resume
Web Framework	FastAPI + Uvicorn	REST API + cursor-based SSE event streaming
Frontend	Single-page HTML (static/index.html)	Document cards, per-doc feedback, PDF export
Web Search	DuckDuckGo Instant Answer API	Company research — no API key required
PDF Parse	pypdf	Extract text from uploaded resume / JD PDFs
PDF Export	fpdf2	Render markdown to downloadable PDF
Evaluation	deepeval (GEval / LLM-as-judge)	Quality and fabrication regression CI gate

8. API Reference

Endpoint	Method	Description
/	GET	Serve single-page web UI
/api/submit	POST	Start agent run; returns session_id
/api/status/{session_id}	GET	Cursor-based event polling for real-time node updates
/api/feedback/{session_id}	POST	Submit global approval or per-document feedback
/api/parse-pdf	POST	Extract text from uploaded resume or JD PDF
/api/export-pdf	POST	Render markdown document to downloadable PDF